



Figure 1: Swagger UI for REST APIs

To retrieve instances matching certain criteria, give the following command:

```
{ "where" : { "temperature": { "gt": "20" } } }
```

To list in an ascending order, type:

```
{ "order" : "temperature ASC" }
```

These filters are encoded in the *URL query string* as *'filter=<url-encoded-criteria>'* with HTTP requests.

You can play around with other REST endpoints and verbs exposed on the current model through this UI, and also observe the equivalent *curl* command usage for making requests using any other client.

Adding a Web template

You can add some static Web content as the home page. For this, edit the *files* section in *server/middleware.json* as follows, to define the static middleware:

```
"files": {
  "loopback#static": {
    "params": "$!../client"
  }
},
```

Then disable *server/boot/root.js* by removing or renaming it with an extension other than *.js*.

Now place any static pages with HTML, CSS and client-side scripting content under the *client* sub-directory, which will serve as a home page for your application.

Authentication and authorisation

LoopBack comes with good support for authentication and access controls through its built-in models — *User*, *AccessToken*, *ACL*, *RoleMapping* and *Role*. To create database schema for these models, add a small procedure in a *.js* file as described in <http://loopback.io/doc/en/lb2/Creating-database-tables-for-built-in-models.html> and execute it once.

To create a new user, do a POST operation on */api/Users* with the following payload:

```
{
  "email": "abc@strongloop.com",
```

```
  "password": "hello123",
  "emailVerified": true
}
```

You may add other properties like *realm* for real name, and *username* in the above payload. Now log in with a POST operation on */api/Users/login* using the payload with a registered email ID and password, which returns a response with *accessToken*, with property name *id* and a *ttl* value for validity. The default validity period for *accessToken* is two weeks.

```
{
  "id": "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
  "ttl": 1209600,
}
```

You can use this generated token for further authentication. In the Swagger UI, this token can be set in the field which appears at the top right corner, which adds a *name-value* pair to query string for authentication. For example, to retrieve the list of instances, the following REST operation is used.

```
GET /api/Weather?access_token=xxxxxxxxxxxxxxxxxxxxxxxx
```

You can perform a POST operation on */Users/logout* to invalidate the token explicitly.



Figure 2: Access token in REST Explorer

ACL rules specify the access privileges on created models for users. Let's deny access to all properties of the current model to all users, initially.

```
lb acl
? Select the model to apply the ACL entry to: Weather
? Select the ACL scope: All methods and properties
? Select the access type: All (match all types)
? Select the role All users
? Select the permission to apply Explicitly deny access
```

Now, let's grant access to any authenticated user:

```
? Select the role : Any authenticated user
? Select the permission to apply : Explicitly grant access
```

You can observe these generated ACL rules under the model-specific JSON file in *common/models*, i.e., *weather.json* in our example. With these rules, you can observe that only authenticated users can perform CRUD operations via REST on the *Weather* model.